Coursework 1 Report for Artificial Intelligence and Intelligent Agents

Group 68 Edinburgh

By Nathaniel Puchalski(H00488698) and

Aaron Thomson(H00469705)

GenAI statement: We used GenAI to explain part of Task 1A to us.

Link to video demo: AI CW1 - group 68 video.mp4

Part 1A:

Variables

Each cell is a variable and to differentiate between each variable we included an identifier called c and r;

$X_{c,r}$ where X is the cell(variable) and c is the column(0-8) and r is the row(0-8)

For example:

- $X_{0,0}$ = top left cell.
- $X_{1,0}$ = cell immediately to its right.
- $X_{1,1}$ = cell directly below the previous variable(cell)

Since Sudoku is a 9x9 grid, there are:

9 times 9 which equals to 81 variables.

0,0	1,0	2,0	…					
0,1	1,1							
0,2								
…								

Domains

$D(X_{c,r}) = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

This means that each empty cell can contain any digit from 1 to 9, provided it satisfies the constraints. If a cell already contains a number given by the puzzle, its domain contains only that number.

For example, if $D(X_{0,0})$ is initially 5:

$D(X_{0,0}) = \{5\}$

Constraints:

- Every row must contain the digits 1 to 9 without repetition.
- Every column must contain the digits 1 to 9 without repetition.
- Each 3x3 sub grid must contain the digits 1 to 9 without repetition.

Worst-case time complexity and effect of backtracking on the search space

The worst-case time complexity of brute-force search is $O(9^m)$, where m represents the number of empty cells in the Sudoku grid. This is because each empty cell can contain one of nine possible digits. For a completely empty Sudoku grid containing 81 cells, the worst-case time complexity is $O(9^{81})$. Backtracking also has a worst-case time complexity of $O(9^m)$. However, the main difference between the two approaches is how much of the search space they explore.

Brute-force search considers all possible combinations of values before checking whether they form a valid solution. In contrast, backtracking checks whether each assigned value satisfies the Sudoku constraints before continuing. If a value violates a constraint, it is immediately rejected, preventing the algorithm from exploring further combinations based on that invalid assignment. This process is known as pruning.

For example, a Sudoku cell initially has nine possible values:
{1,2,3,4,5,6,7,8,9}

If the cell's row already contains 1, 2 and 3, its column contains 4 and 5, and its 3x3 sub grid contains 6 and 7, the remaining valid values are:
{8,9}

In this situation, backtracking only needs to consider two values instead of nine. If an assigned value eventually leads to a contradiction, the algorithm returns to a previous cell and attempts a different value. By pruning invalid branches early, backtracking reduces the number of combinations explored and therefore improves practical efficiency compared to brute-force search. However, despite this reduction in the search space, its worst-case time complexity remains exponential at $O(9^m)$.

Part 1B:

The chosen approach that we have taken is a Constraint Satisfaction Problem (CSP) with plain backtracking. We chose this approach as we had already worked out the variables and domain for the sudoku if it was a CSP in part A of this task. The reasoning behind choosing plain backtracking was that the more complex methods of backtracking required much more computation and foresight into which variables were assigned, therefore increasing the runtime of the program. We believed that plain backtracking would be the most efficient way to solve given sudokus. The evidence through testing proved to us that we were incorrect in thinking this.

The approach that was taken was that the program would first navigate to the first empty square in the 2D array containing the sudoku. It would then assign a valid value to this square (a value that satisfies the given constraints). Then the program would move onto the next square, continually assigning valid values until there are no empty squares left, or there are no valid values that can be assigned. In the case of there being no valid values, the program would then backtrack, changing previous values until the sudoku can be solved.

The three sudokus we have tested the sudoku solver with are the three difficulties of sudoku on the New York Times website. The way that we decided to test the efficiency of our program was through tracking the runtime and through counting how many times the program had to backtrack. The sudokus we have used are from 24/10/26. We have easy, medium and hard:

Easy

1	4	7				6		
		8	6	7		1	2	
	6		3	1		8		7
8					7	5	4	9
4	7	1			8			
	3			2	6			
		4	2		9		6	
6	1	2				9	8	
9			5	6				

Medium

9								
			8		9	1	2	
	1				6	3	8	
8		3		6	7			
		5	1					6
			4					
	4							3
	2	7						
				4			6	7

Hard

	9	6			2			
4	3			8				
				7	5		6	
							9	5
6		9	4				7	
8								
			1	6				
9				2			4	7
3		2	7					

To calculate the execution time for each of the sudokus, we will run the program 10 times with each sudoku and take an average time. We start with the easy sudoku, the

average execution time of our program with the easy sudoku is 73.7 milliseconds. For the medium sudoku it is 79.1 milliseconds, and for the hard sudoku it is, 81.4 milliseconds. Although, a better way to measure how good our CSP is for this problem is by counting the number of times that the program had to backtrack. For the easy sudoku it only had to backtrack 45 times. For the medium sudoku it had to backtrack 7851 times, and for the hard sudoku it had to backtrack 9464 times.

The approach we have taken is advantageous as the program runs quickly and always either displays the solved sudoku to the user or tells the user that it is not possible to solve the sudoku when required. Alternative CSP techniques such as forward checking would increase the efficiency of the program as it would lead to the program backtracking less.

To formulate a sudoku as an A* search problem, the state representation would be the current state of the entire sudoku. The actions would be assigning valid numbers to empty squares. The path cost would be the number of assignments that have been made so far. A possible heuristic that could be used is the number of squares that are still needing to be assigned.

We believe that the approach that we have taken is more suited to solving a sudoku than an A* approach. This is because our approach ventures down only one pathway at a time and doesn't consider any others until it needs to backtrack, whereas an A* approach considers all possible pathways from the current state of the sudoku and then picks the most promising one, therefore requiring more computations and using up more space.

Part 2A & B:

2.1 Object Types and Predicates

Five main object types were defined; rover, lander, waypoint, data and sample. Rovers perform exploration tasks, Landers receive data and store samples, and waypoints represent locations on the lunar surface. The data type is used for both images and scans because they share the same Rover memory capacity. As, a Rover can only store one image or scan at a time.

Several predicates represent the state of the mission. The at and lander-at predicates identify Rover and Lander locations, while belongs-to specifies which Lander a Rover is assigned to. The aboard and deployed predicates distinguish between Rovers that are still inside their Landers and those operating on the surface.

The connected predicate represents the directed paths between waypoints. For example, a connection from WP1 to WP2 does not automatically allow movement from WP2 to WP1. The reverse connection must be defined separately. Other predicates track scientific tasks. image-at and scan-at represent data available at waypoints, while sample-at represents the location of a sample. The predicates data-transmitted and sample-stored indicate that scientific objectives have been completed.

2.2 Action Schemas

The domain contains seven actions; “deploy”, “move”, “capture-image”, “perform-scan”, “transmit-data”, “collect-sample” and “store-sample”.

The “deploy” action requires a Rover to be aboard its assigned Lander. “Deploy” places the Rover at the Lander's waypoint, marks it as deployed and removes its aboard status. The “move” action requires the Rover to be deployed and a directed connection to exist between its current waypoint and destination. Its effects remove the previous location and assigns the new location.

The “capture-image” and “perform-scan” actions requires the Rover to be at the correct waypoint with free memory. After collecting data, the Rover's memory becomes occupied. The “transmit-data” action requires the Rover to be carrying the specified

data to be able to transmit it to its assigned Lander. Transmission marks the data as successfully delivered and frees the Rover's memory.

The “collect-sample” action requires the Rover to be at the sample's waypoint and have available carrying capacity. The “store-sample” action requires the Rover to return to its assigned Lander's waypoint while carrying a sample. It also checks that the Lander has available storage.

2.3 Resource and Storage Constraints

The domain models Rover memory using the predicates “memory-free” and “carrying-data”. A Rover can capture an image or perform a scan only when “memory-free” is true. Collecting data removes this predicate and records the specific data item using “carrying-data”. This prevents a Rover from collecting another image or scan while its memory is occupied. Once the data is transmitted to its assigned Lander, “carrying-data” is removed and “memory-free” becomes true again.

“Sample-carrying” capacity is modelled separately using “sample-capacity-free” and “carrying-sample”. Collecting a sample removes the Rover's available sample capacity, preventing it from collecting a second sample. Storing the sample fills this capacity.

Lander storage is controlled using “lander-storage-free”. When a sample is stored, this predicate is removed and sample-stored becomes true. This ensures that each Lander can store at most one sample. The mission goals use “data-transmitted” and “sample-stored” rather than simply checking whether a Rover has collected data or picked up a sample. Therefore, the planner must complete the required delivery or storage actions to achieve the goals.

2.4 Mission Implementation and Testing

The PDDL domain was tested throughout development by repeatedly running the planner and examining the generated plans. During implementation, errors were identified and corrected before running the planner again. This iterative process helped ensure that the domain actions, preconditions and effects worked as intended.

The domain was tested using Missions 1 and 2. Mission 1 involved one Rover and one Lander, while Mission 2 involved two Rovers and two Landers. Both missions used the same domain, showing that the action schemas could be reused without modification.

Both missions were tested multiple times. The plans were then compared directly against the coursework mission goals to confirm that all the goals were achieved. This included checking that images and scans were transmitted to the correct Landers and that samples were physically stored in their assigned Landers. The plans were also made sure that Rover movements followed the directed waypoint connections and that actions were performed in a valid sequence.

Part 2C:

2.5 Astronaut Extension

Three new predicates were introduced; “astronaut-belongs-to”, “in-docking-bay” and “in-control-room”. These represent each astronaut's assigned Lander and current internal location. Two new actions, “move-to-control-room” and “move-to-docking-bay”, allow astronauts to move between the two areas of their own Lander. Their preconditions check the astronaut's Lander assignment and current room, while their effects update the astronaut's location. This prevents astronauts from moving between different Landers. The existing “deploy”, “transmit-data” and “store-sample” actions were changed to include an astronaut parameter and additional preconditions.

The astronaut extension adds new conditions that must be satisfied before certain Rover actions can take place. Deployment and sample storage are only possible when an astronaut is in the docking bay, where data transmission requires an astronaut in the control room. As Bob begins in the control room and Alice in the docking bay, they need to change rooms to assist with different tasks throughout the mission. Actions like Rover movement, image capture, scanning and sample collection remain unaffected by these changes.

We used Microsoft Visual Studio Code to write the code for task 2. The extension we used was “PDDL” by Jan Dolejsi. The planner used for task 2 was “BFWS –FF-parser version”.